Proiect

Dezvoltarea aplicațiilor Internet

Agregator de știri

Efectuat de: Călin Cătălina
Lupu Cristian

București, 2021

Cuprins:

1. Introducere
2. Proiectare
3. Dezvoltare
4. Testare
5. Instalare
6. Utilizare

1. Introducere

Proiectul nostru a presupus dezvoltarea unui site Web de tip agregator de știri cu frontend si backend de la 0, în care utilizatorul poate sorta noutățile în funcție de preferințele sale (după tag-uri, după sursa etc).

2. Proiectare

2.1 Frontend

Interfaţa cu utilizatorul este parte a aplicaţiei software care permite utilizatorilor să-şi exprime intenţiile de operare asupra aplicației şi să interpreteze rezultatele acţiunilor efectuate de aceasta. Utilizatorul decide dacă interfaţa folosită îi este cu adevărat utilă, iar componentele hardware şi software pentru el sunt doar unelte cu ajutorul cărora interfaţa e construită.

Interfața este percepută nu doar ca parte vizuală a software-ului, deoarece din punctul de vedere al utilizatorului, reprezintă întregul sistem, deci trebuie sa fie plăcută, intuitivă, elementele să fie ergonomic organizate, ușor de folosit. Din punct de vedere al programatorului, ea trebuie să fie ușor de dezvoltat, în acest scop am folosit un framework despre care vom vorbi în capitolul despre dezvoltare.

Partea de proiectare a interfeței a constat din înţelegerea nevoilor utilizatorilor, care intră pe un site de știri pentru a se recrea și pentru a afla ceva nou, și consta ,la fel, din proiectarea (designul) . Design-ul trebuie să fie minimalist, intuitiv și ușor de utilizat.

Pentru a proiecta interfața utilizatorului , și anume design-ul pe care îl vom folosi, am folosit aplicația Balsamiq. Balsamiq Wireframes este o aplicație UX pentru utilizatori ce doresc să creeze mock-up-uri(wireframe-uri) pentru aplicația pe care urmează să o dezvolte. Acesta permite proiectantului să aranjeze widget-uri preconstruite folosind un editor drag-and-drop. Aplicația este oferită într-o versiune desktop, precum și un plug-in pentru Google Drive, Confluence și JIRA.

Mai jos, prezentăm câteva mock-up-uri pentru diverse pagini din aplicația noastră.

Pagina principală:

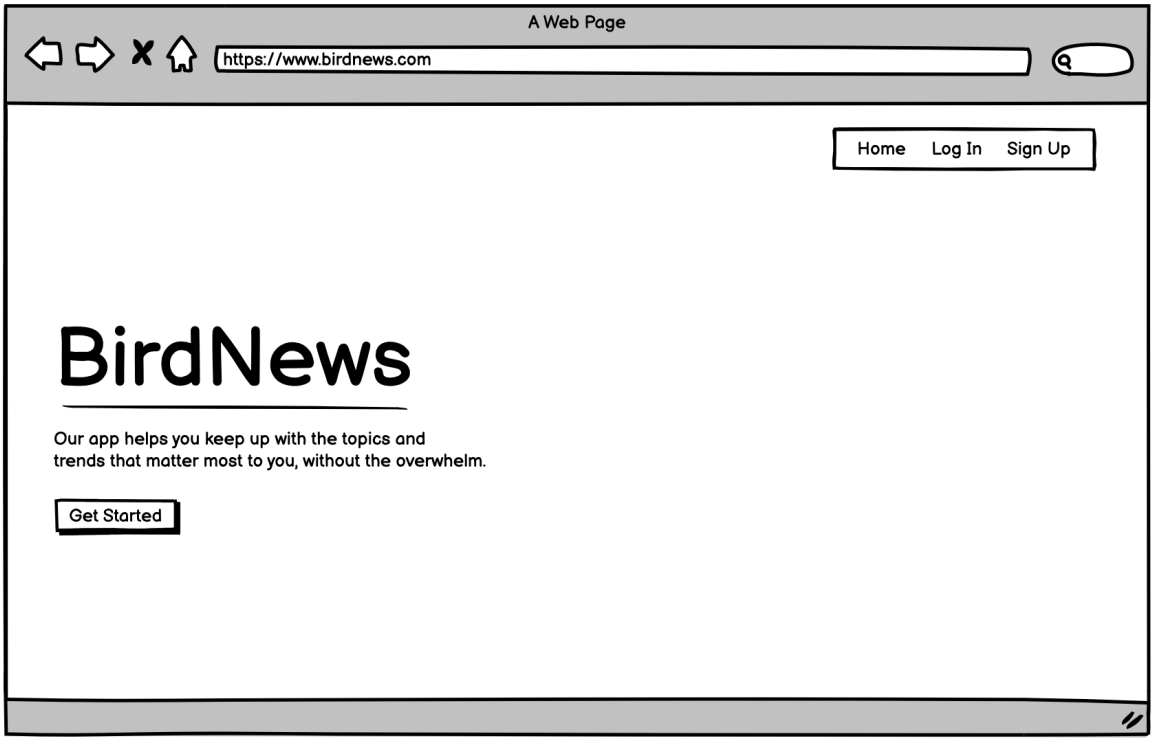


Figura 1. Mock-up Balsamiq pentru pagina principală

Pagina pentru înregistrare utilizator:

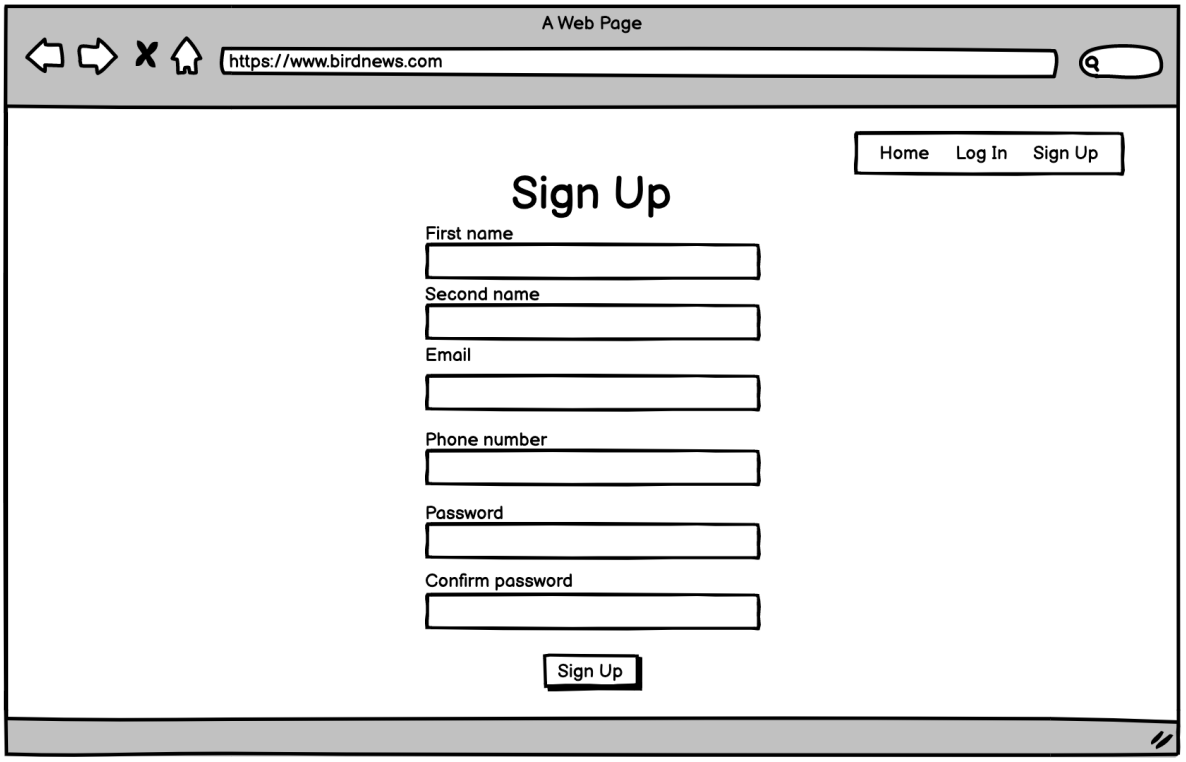


Figura 2. Mock-up Balsamiq pentru pagina de înregistrare

Pagina pentru autentificare utilizator:

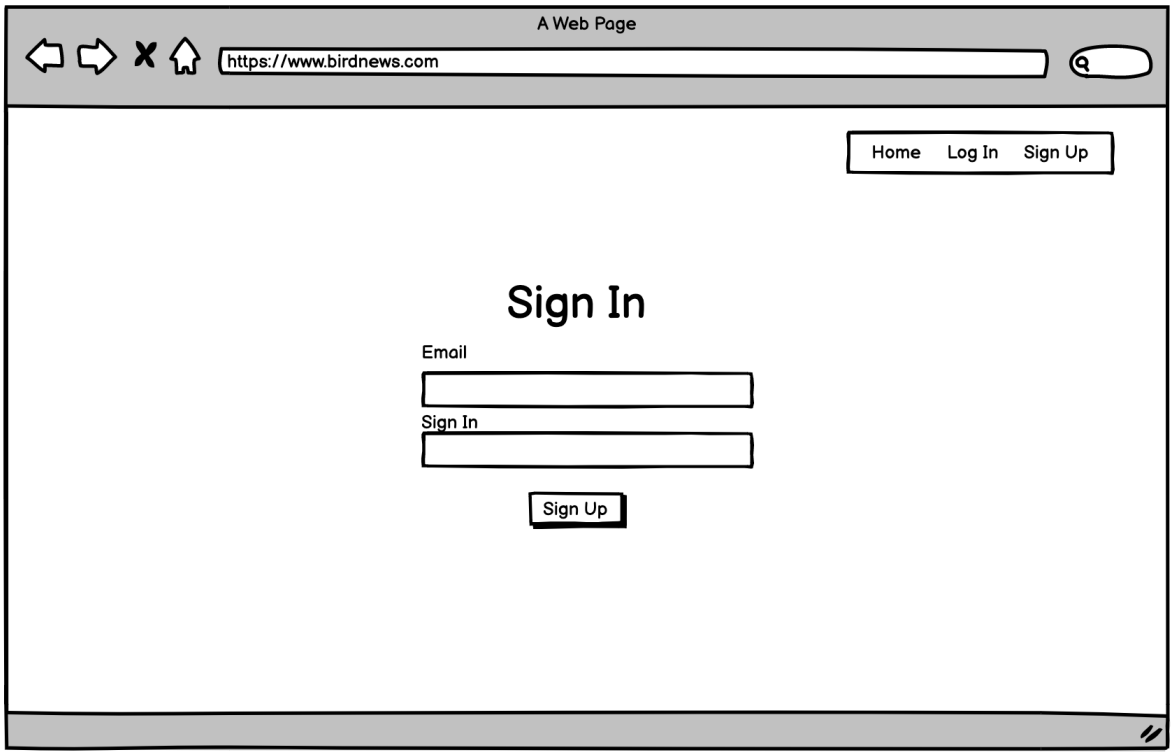


Figura 3. Mock-up Balsamiq pentru pagina de autentificare

Pagina pentru pagina de știri:

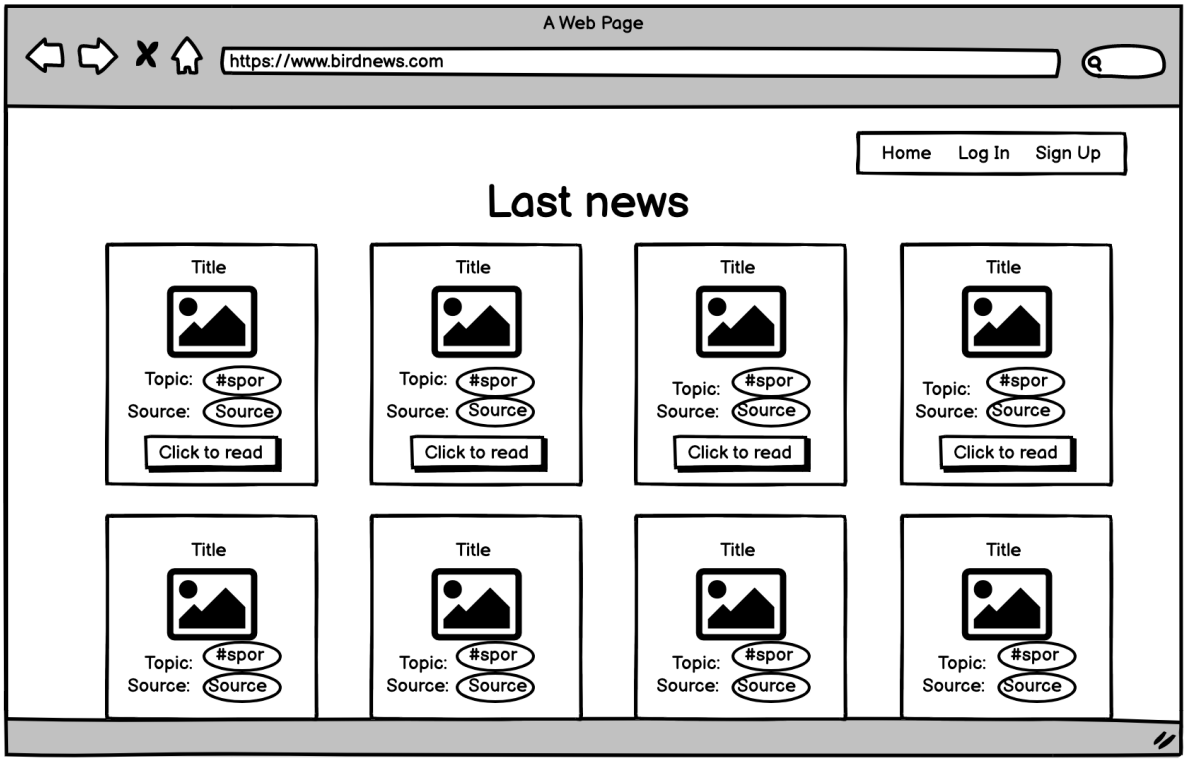


Figura 4. Mock-up Balsamiq pentru pagina cu știri

Pop-up cu o știre selectată:

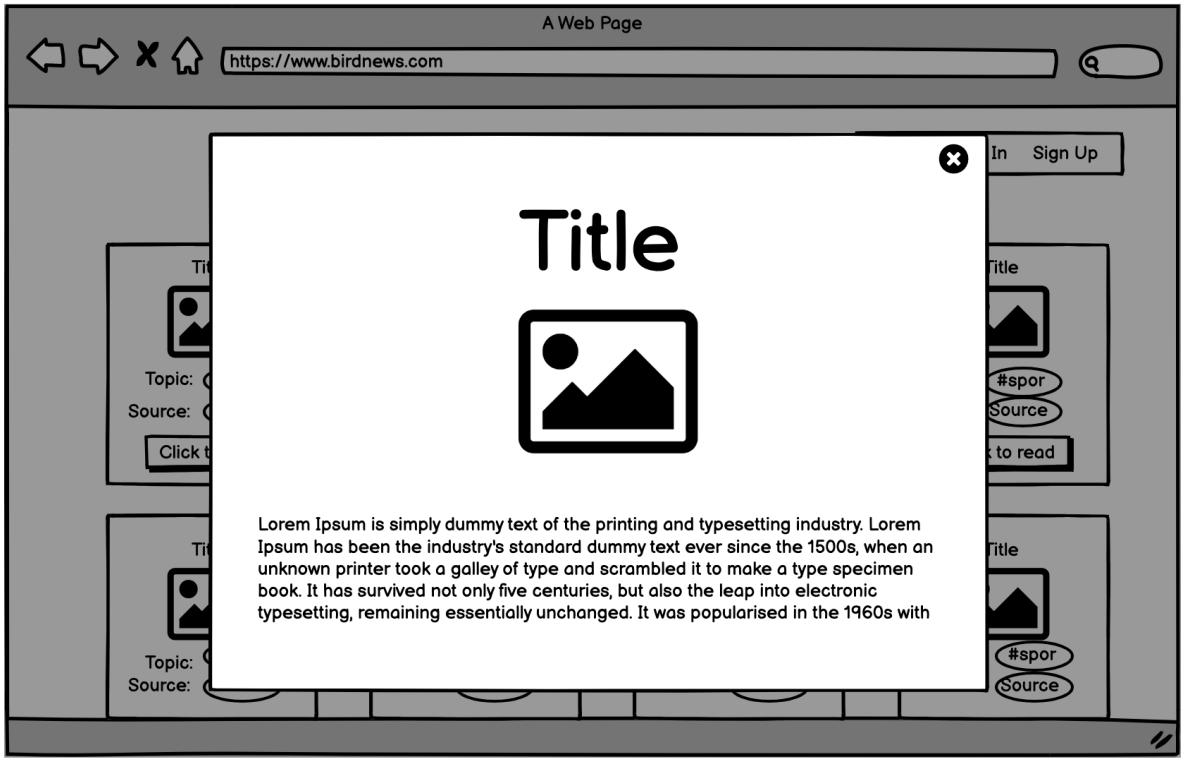


Figura 5. Mock-up Balsamiq pentru pop-up cu o știre selectată

Pentru a proiecta interfața cu utilizatorul, am urmat câțiva pași și anume identificarea și înţelegerea nevoilor utilizatorilor finali, analiza task-urilor și contextului interacţiunilor utilizator-aplicație, prototipizarea interfeţei, evaluarea interfeţei, programarea interfeţei și iterarea etapelor anterioare.

2.2 Backend

Backend-ul este o altă componentă importantă în cadrul aplicației noastre, întrucât face conexiunea dintre interfața utilizatorului și baza de date. Layer-ul de backend este împărțit în 3 servicii:

- Serviciul de utilizatori, care reprezintă un REST API ce expune endpoint-urile necesare pentru înregistrarea, logarea, delogarea și actualizarea utilizatorilor.
- Serviciul de noutăți, care reprezintă un REST API ce permite accesarea noutăților din baza de date.
- Serviciul de Web Scraping, care este un task planificat ce rulează o dată la 10 minute pentru a extrage datele din sursele indicate în baza de date.

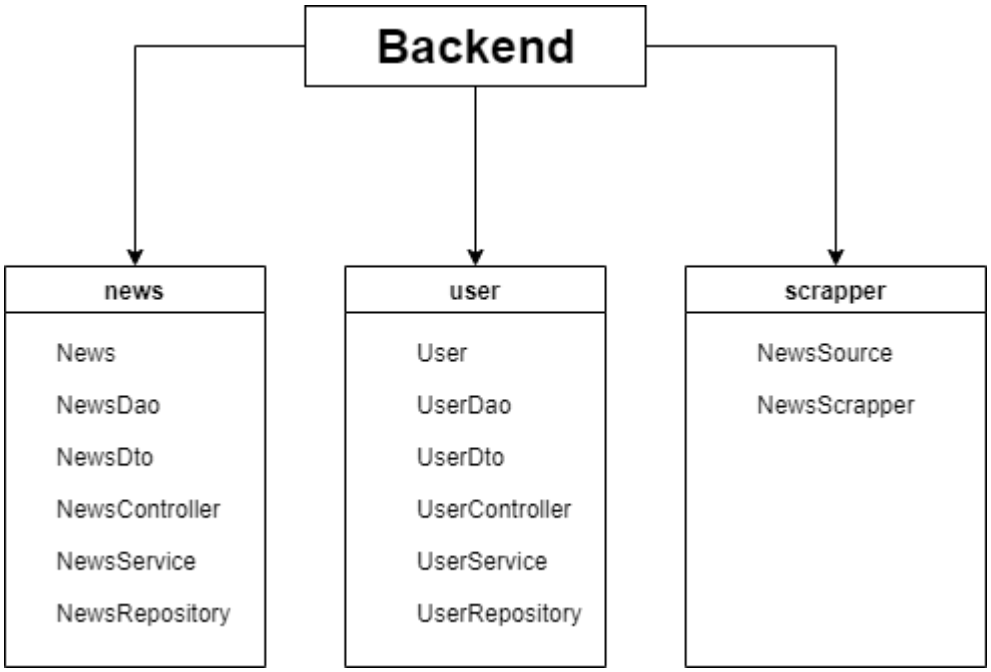


Figura 6. Arhitectura serviciilor backend

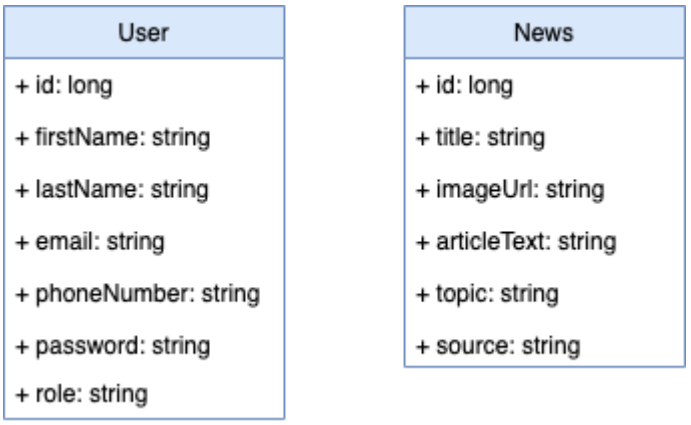
2.3 Baza de date

Proiectarea bazei de date am efectuat-o după stabilirea entitatilor necesare pentru dezvoltarea aplicației.

Astfel am identificat că avem nevoie de 3 entități:

- utilizator (User) : care contine campuri ca prenume(firstName), nume(lastName) , email(email), telefon(phoneNumber), parola(password), rol(role), id(id).

- stire(News) : care contine campuri ca id(id), titlu(title), adresa imaginii asociate(imageUrl), textul articolului (articleText), topic (topic), sursa de unde a fost preluata noutatea (source).



3. Dezvoltare

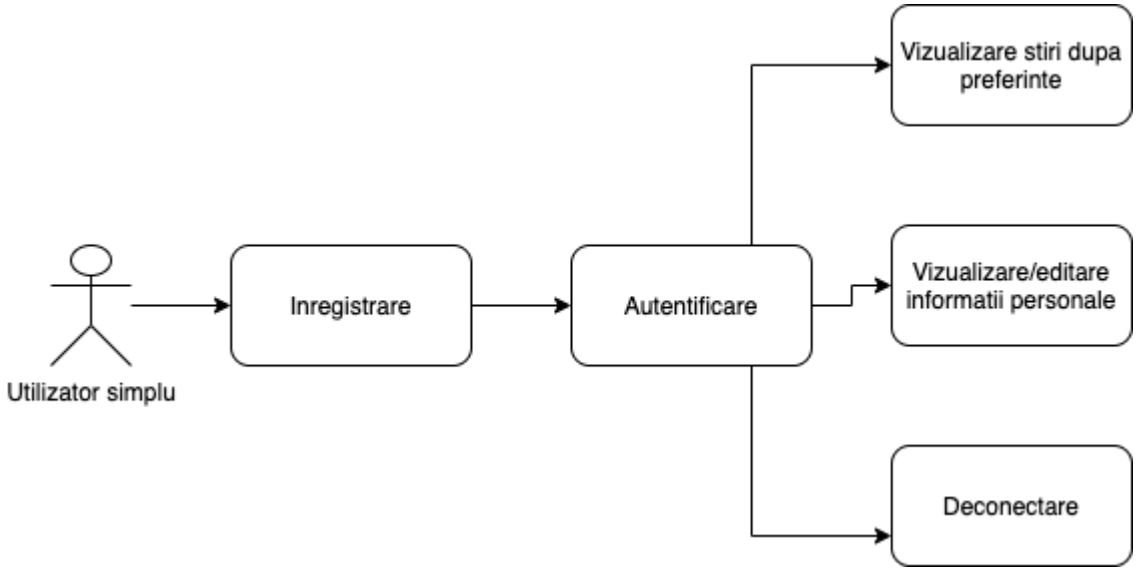


Figura 7. Diagrama functionalitati utilizator simplu

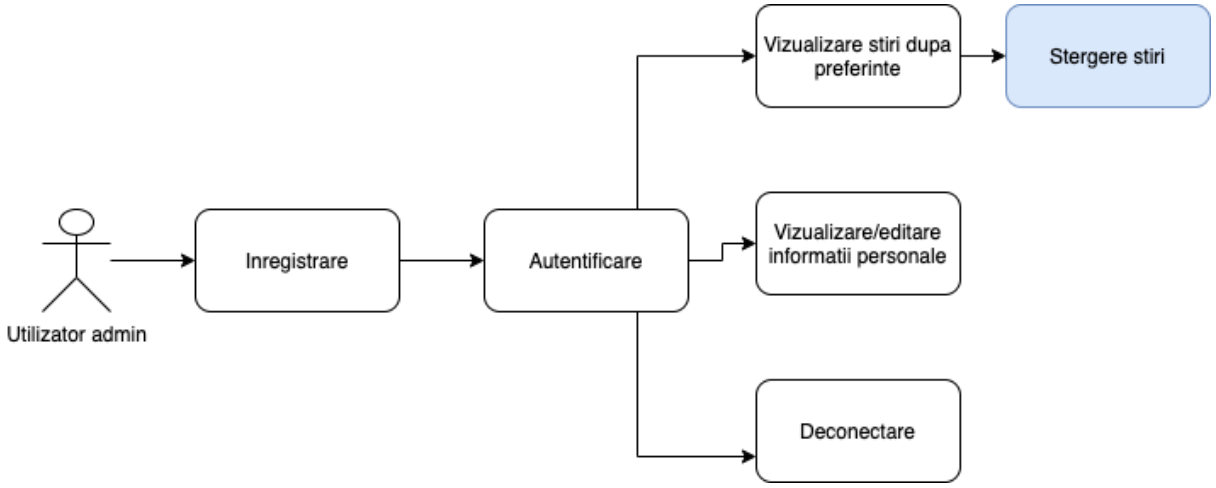


Figura 8. Diagrama functionalitati utilizator admin

3.1 Frontend

Pentru a implementa cu rapiditate și ușurință interfața grafică a aplicației am folosit framework-ul Angular. **Angular** este un framework dezvoltat de Google folosit pentru construirea de SPA (Single Page Application) în HTML, CSS și JavaScript sau mai nou TypeScript (care este compilat în JavaScript). Dacă nu ai lucrat până acum cu TypeScript, nu te speria. Îți voi arăta elementele de bază într-un alt tutorial.

Acesta are o imensă comunitate de programatori unde poți pune întrebări sau găsi răspunsuri la problemele pe care le întâmpini.

În același timp este și foarte folosit de companii, existând o mulțime de job-uri disponibile cu salarii foarte bune. Acum că știi aceste lucruri, sunt convins că ai înțeles ce este Angular și că este o alegere excelentă să îl înveți. Spre deosebire de Angular care are un cod curat, ușor de întreținut și testat.

Practic Angular îți face viața mai ușoară. Este adevărat că îți va lua ceva timp până înveți Angular, dar dacă ai răbdare vei construi aplicații mai repede și ușor.

În al doilea rând, npm colecționează toate pachetele (librării, framework-uri, aplicații, etc) pe care altcineva le-a scris și le-a publicat pe npm (sub licențe open-source, desigur). Desigur, nu toate pachetele sunt revoluționare, dar majoritatea îndeplinesc o anumită funcție sau chiar multiple funcționalități. Astfel, noi nu mai trebuie să stăm și să reinventăm roata de fiecare dată când ne apucăm de un proiect nou. Vrem să lucrăm cu date sau ore, putem să folosim moment.js. Vrem să facem un site responsive, și să îi adăugăm și un pic de culoare? Instalăm bootstrap. Vrem să facem grafice? probabil chart.js. Mai sunt multe alte puncte forte, precum o varietate mare de librării și pachete dezvoltate special pentru Angular, care rezolvă anumite probleme (Angular Material, Angular Fire, AngularCommerce, ngrx, nx/nrwl, etc), sau modularitatea cu care dezvolți în Angular, care îți permite o reutilizare eficientă a codului. Pentru un design mai atractiv am folosit **Angular Material**.

Partea de front-end a fost totalmente dezvoltată de Calin Catalina. Task-uri efectuate de Calin Catalina:

- crearea proiectului Angular
- proiectarea front-end
- crearea mock-up-urilor interfetei grafice
- dezvoltarea paginii principale + design css pagina principala
- dezvoltarea paginii înregistrare utilizatorilor + design css înregistrare utilizatorilor
- dezvoltarea paginii autentificare utilizatorilor + design css autentificare utilizatorilor
- dezvoltarea paginii cu știri + design css paginii cu știri
- dezvoltarea paginii profilul meu + design css paginii profilul meu + pop-up pentru editarea datelor utilizatorului

- importarea tuturor componentelor necesare
- adaugarea termenilor si conditiilor ce trebuie acceptate de utilizator
- adaugarea serviciilor necesare pentru interacțiunea componentelor
- efectuarea request-urilor către API (backend)
- testarea interfeței grafice

3.2 Backend

Pentru a implementa partea de backend, am ales limbajul de programare Java, și în special framework-ul Spring. Acesta este un framework open source lansat pentru prima oara in 2003. Astazi, Spring este foarte popular si utilizat in proiecte care folosesc atat JavaEE cat si JavaSE. Spring Framework este o source extension (features), necesara pentru dezvoltarea aplicatiilor de business complexe.

Spring Framework poate sa fie vazuta ca o colectie de framework-uri mai mici, majoritatea putand sa opereze independent de celelalte. Insa, ofera mai multa functionalitate atunci cand sunt folosite impreuna. Abstract classes, factories si beans sunt construite in asa fel incat programatorul sa se poata concentra pe logica de business.

Pentru dezvoltarea rapidă a aplicației bazate pe Spring, a fost folosit Spring Boot, care oferă un server Tomcat încadrat și dependințele necesare pentru o configurație de bază.

Pentru web scraping, a fost folosit biblioteca open-source Jsoup. Aceasta este utilizată în principal pentru extragerea datelor din HTML. De asemenea, permite manipularea și extragerea datelor din documente HTML. Aceasta are o linie de dezvoltare constantă, o documentație excelentă și un API fluent și flexibil. Cu ajutorul acestei biblioteci am atins următoarele scopuri:

- Încărcare: preluarea și analizarea codului HTML
- Filtrare: selectarea datelor dorite și parcurgerea acestora
- Extragerea: obținerea atributelor, textului și HTML-ului nodurilor
- Modificare: adăugarea / editarea / eliminarea nodurilor și editarea atributelor acestora

Task-uri efectuate de Lupu Cristian:

4. Setarea layer-ului de backend
5. Dezvoltarea REST API-urilor pentru utilizatori și știri
6. Dezvoltarea serviciului de web scraping
7. Scrierea testelor unitare și de integrare
8. Configurarea imaginilor de docker pentru frontend, backend și bază de date
9. Proiectarea modelelor de date

3.3 Baza de date

Pentru baza de date am ales PostgreSQL, care este un sistem de baze de date relaționale. Este disponibil gratuit sub o licentă open source de tip BSD. PostgreSQL nu este controlat de nici o companie, își bazează dezvoltarea pe o comunitate răspândită la nivel global, precum și câteva companii dezvoltatoare. PostgreSQL încearcă să se conformeze standardului SQL. Pe lângă faptul că este **liber și open source**, PostgreSQL este extrem de extensibil. De exemplu, puteți să vă definiți propriile tipuri de date și să creați funcții personalizate. Puteți chiar să scrieți **coduri din diferite limbaje de programare** fără a recompila baza de date. Mai multe despre PostgreSQL găsiți aici. Cu alte cuvinte, MySQL este doar marca unui software de bază de date. Același lucru este valabil și pentru **PostgreSQL**.

Limitări

Câteva limitări generale

- Dimensiunea maximă a bazei de date: nelimitat
- Dimensiunea maximă a unei tabele: 32 TB
- Dimensiunea maximă a unei înregistrări: 1,6 TB
- Dimensiunea maximă a unui câmp: 1 GB
- Număr maxim de înregistrari într-o tabelă: nelimitat
- Număr maxim de coloane într-o tabelă: 250 - 1600 în funcție de tipul coloanelor

Pentru a face legătura dintre backend(API) și baza de date am folosit Java Database Connectivity (JDBC), care este o interfață de programare a aplicației (API) pentru limbajul de programare Java, care definește

modul în care un client poate accesa o bază de date. Este o tehnologie de acces la date bazată pe Java utilizată pentru conectivitatea bazei de date. Face parte din platforma Java Standard Edition, de la Oracle Corporation. Oferă metode de interogare și actualizare a datelor dintr-o bază de date și este orientat spre baze de date relaționale.

4. Testare

4.1 Frontend

Pentru a testa componentele interfeței grafice, am folosit Cypress. Cypress este un instrument de testare front-end de nouă generație construit pentru web-ul modern. Se adresează problemelor cheie cu care se confruntă dezvoltatorii și inginerii QA atunci când testează aplicații moderne. Cypress vă permite să scrieți toate tipurile de teste: teste end-to-end, teste de integrare, teste unitare. Cypress poate testa orice rulează într-un browser. Testele scrise în Cypress sunt menite a fi ușor de citit și de înțeles. Cypress rulează la fel de rapid pe cât browserul dvs. poate reda conținut. Se pot urmări testele rulate în timp real pe măsură ce sunt dezvoltate aplicațiile. Cypress este o aplicație desktop instalată pe computer.

4.2 Backend

Pentru a testa serviciile de backend sunt prezente teste unitare care rulează izolat, precum și teste de integrare care încarcă contextul Spring înainte de executarea testelor. Pentru scrierea testelor au fost folosite framework-urile Mockito și JUnit.

Testele de integrare au fost păstrate separat față de testele unitare și sunt rulate separat. Acest lucru este făcut folosind un profil diferit pentru a rula doar testele de integrare. Câteva motive pentru a face acest lucru este faptul că testele de integrare necesită mult timp și au nevoie de o bază de date reală pentru a fi executată.

10. Instalare

Pentru a instala proiectul pe mediul local, trebuie urmăriți pașii de mai jos:

1. Asigurați-vă că aveți dependințele necesare deja instalate:

    - versiune git >= 2.31.1
    - versiune docker >= 20.10.6
2. Clonați repository-ul proiectului pe mediul local:
    - git clone https://github.com/crlupu/news-aggregator.git
3. Executați comenzile pentru crearea și executarea serviciilor aplicației:
    - cd news-aggregator
    - docker-compose build
    - docker-compose up
4. Puteți accesa aplicația navigând în orice browser la adresa http://localhost:4200/